

# CH-16 Deep Learning

# Introduction

- Deep learning is a powerful subset of machine learning.
- It has achieved remarkable success in computer vision and other fields.
- Its growth is fueled by:
  - ◇ Big data availability
  - ◇ High processing power
  - ◇ Faster Internet speeds
  - ◇ Advances in parallel computing (hardware & software)

## Keras and TensorFlow

- Scikit-learn allows simple model creation.
- **TensorFlow** is an open-source deep learning library developed by Google.
- Deep learning needs connecting multiple layers.
- **Keras**: high-level interface to TensorFlow (Google).
- Keras is a high-level API (Application Programming Interface) for building and training neural networks.
- Developed by François Chollet (Google Mind team).
- **His book**: Deep Learning with Python.
- Google runs 1000+ TensorFlow and Keras projects.

## Models

- Deep learning is complex, with heavy math.
- We avoid deep math, focus on clear English.
- Keras → Deep Learning Scikit-learn → Machine Learning.
- Define, parameterize, and use components (object-based programming).

## Experiment with Your Models

- ML/DL are empirical.
- You tweak models to improve performance.
- Keras supports easy experimentation.

## Dataset Sizes

- DL works best with lots of data.
- Also useful for small data using:
  - ◇ **Transfer learning** – reuse knowledge from prior models.
  - ◇ **Data augmentation** – expand data using transformations (e.g., image rotation).

## Deep Learning Applications

- Deep learning is being used in a wide range of applications, such as:
  - Game playing
  - Computer vision:
    - Object recognition
    - Pattern recognition
    - Facial recognition
  - Self-driving cars
  - Robotics
  - Improving customer experiences
  - Chatbots
  - Diagnosing medical conditions
  - Google Search
  - Facial recognition
  - Automated image and video captioning
  - Enhancing image resolution
  - Speech recognition
  - Language translation
  - Predicting election results

## Keras Built-In Datasets:

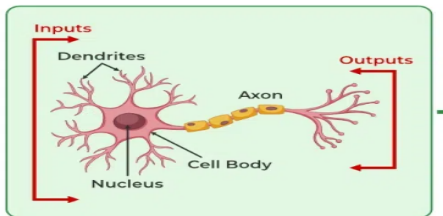
- Keras offers built-in datasets from `tensorflow.keras.datasets`.
- Common datasets used in examples, exercises, and projects:  
**MNIST**: 28×28 grayscale digit images (0–9), 60k train / 10k test.  
<http://yann.lecun.com/exdb/mnist/>.

## 16.4 Neural Networks

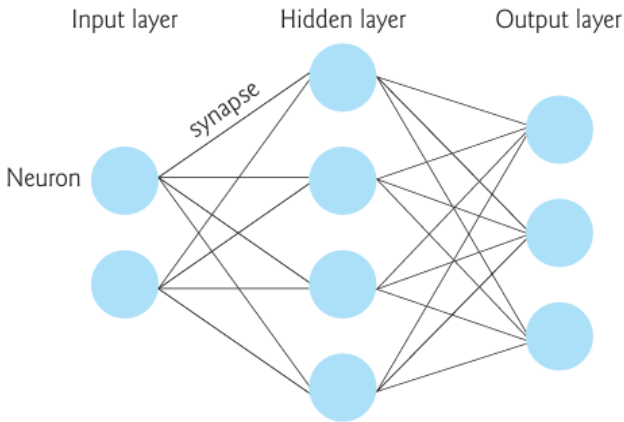
- Deep learning is a form of machine learning that uses artificial neural networks to learn.
- Deep learning uses artificial neural networks (ANNs) to learn patterns from data.
- ANNs are inspired by the biological brain: **neurons** communicate via **synapses**.
- Learning strengthens neural pathways to improve performance on tasks.

### Artificial Neurons

- Interconnected artificial neurons simulate brain activity for learning.
- In supervised learning, the goal is to predict target labels.
- Neural networks are trained on labeled data and make predictions on unseen data.



- A typical ANN has:
  - **Input layer:** receives raw features
  - **Hidden layer(s):** learn intermediate features
  - **Output layer:** generates predictions
- **Fully connected:** every neuron in a layer connects to every neuron in the next layer.



# Learning is an Iterative Process

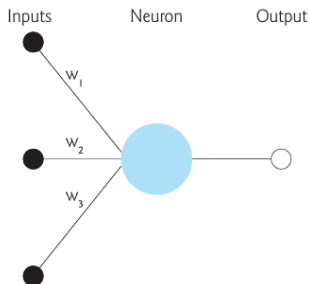
- Learning is gradual, like a baby learning to walk—built on smaller skills and feedback.
- Neural networks also learn iteratively over time through training.
- **Epoch:** One full pass through the training dataset.
- Number of epochs is a **hyperparameter**; it affects training quality and must be tuned.
- Inputs to the model are the **features** from the training data.
- Hidden layers learn and extract features from inputs.
- Output layer interprets these features to make predictions.
- Feedback (loss) is used to update the model in each epoch.

# How Artificial Neurons Decide to Activate Synapses

- Each neuron receives inputs from previous layer, each multiplied by a corresponding **weight** ( $w_1$ ,  $w_2$ ,  $w_3$ , etc.).
- The weighted inputs are summed and passed through an **activation function**.
- The activation function decides whether to pass a signal to the next layer.
- Weights are initially set randomly when training starts.
- During training, the network adjusts weights to minimize the **loss** (prediction error).
- **Loss function** computes how far the prediction is from the actual output.
- The network uses **backpropagation** to distribute the loss back through the layers.
- Weights are optimized gradually using **gradient descent**.

# Perceptron: The Basic Unit of a Neural Network

- A **perceptron** is a single artificial neuron.
- **Takes multiple inputs**  $x_1, x_2, \dots, x_n$ .
- Each input has an associated weight  $w_1, w_2, \dots, w_n$ .
- **Computes weighted sum:**  $\sum_{i=1}^n w_i x_i + b$  (where  $b$  is bias).
- Passes the sum through an **activation function**  $f(\cdot)$ .
- **Output:**  $y = f\left(\sum w_i x_i + b\right)$
- **Activation functions:** Step, Sigmoid, ReLU, etc.
- Used to build layers in neural networks.



# Perceptron Architectures

## 1. Single-Layer Perceptron (SLP):

- Computes output directly from weighted sum and activation:

$$\hat{y} = f \left( \sum_{i=1}^n w_i x_i + b \right)$$

- Suitable for linearly separable problems (e.g., AND, OR).

## 2. Multi-Layer Perceptron (MLP):

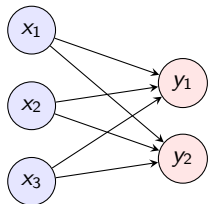
- Has one or more hidden layers:

$$a^{(1)} = f(W^{(1)}x + b^{(1)}), \quad \hat{y} = f(W^{(2)}a^{(1)} + b^{(2)})$$

- Enables learning of non-linear decision boundaries.

# SLP vs MLP Architecture (Labeled)

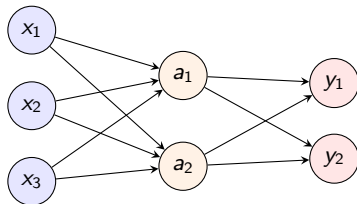
**SLP**



Input

Output

**MLP**



Input

Hidden

Output

# MLP Architecture and Notation

- Input:  $\mathbf{x} \in \mathbb{R}^n$
- Hidden layer with  $l$  neurons:

$$\mathbf{a}^{(1)} = f\left(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\right)$$

- Output layer with  $m$  neurons:

$$\hat{\mathbf{y}} = f\left(\mathbf{W}^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)}\right)$$

- $\mathbf{W}^{(1)} \in \mathbb{R}^{l \times n}$ ,  $\mathbf{W}^{(2)} \in \mathbb{R}^{m \times l}$

# Activation Function and Loss Function

- Common activation functions:

**Sigmoid:**  $\sigma(z) = \frac{1}{1+e^{-z}}$

**ReLU**(Rectified Linear Unit):  $\text{ReLU}(z) = \max(0, z)$

- Loss function for classification (e.g., **cross-entropy**):

$$\mathcal{L} = - \sum_i y_i \log(\hat{y}_i)$$

- Loss function for regression (e.g., **MSE**):

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- Softmax function:**

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}.$$

# Gradient Descent and Backpropagation

- **Backpropagation** is the process of updating the weights and biases of a neural network by calculating the gradient of the loss function with respect to each parameter (weights and biases).
- The **gradients** indicate how much the parameters need to change in order to reduce the error (loss) between the predicted output and the actual target output.
- Update rule for weights:

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{W}}$$

- Update rule for biases:

$$\mathbf{b} \leftarrow \mathbf{b} - \eta \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{b}}$$

- $\eta$  is the learning rate (e.g., 0.01)
- Gradients are computed via backpropagation using the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(2)}} = \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{y}}} \cdot \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{W}^{(2)}}$$

# Training Steps of MLP

- Initialize weights  $\mathbf{W}^{(1)}$ ,  $\mathbf{W}^{(2)}$  and biases randomly.
- Forward pass to compute output.
- Compute loss using ground truth.
- Backpropagate to compute gradients.
- Update weights and biases.
- Repeat for several epochs.

# Feedforward ANN Architecture: Classifying the Iris Dataset

## Layers:

- **Input Layer:** 4 neurons (one for each input feature)
- **Hidden Layer:** 10 neurons (with ReLU activation)
- **Output Layer:** 3 neurons (with Softmax activation for classification)

## Hidden Layer Computation:

$$a^{(1)} = f(W^{(1)}x + b^{(1)})$$

Where:

- $x \in \mathbb{R}^4$
- $W^{(1)} \in \mathbb{R}^{10 \times 4}$
- $b^{(1)} \in \mathbb{R}^{10}$
- $f$  is the ReLU activation function

## Output Layer Computation:

$$\hat{y} = \text{softmax}(W^{(2)}a^{(1)} + b^{(2)})$$

Where:

- $W^{(2)} \in \mathbb{R}^{3 \times 10}$
- $b^{(2)} \in \mathbb{R}^3$

## Loss Function (Categorical Crossentropy):

$$L = - \sum_{i=1}^3 y_i \log(\hat{y}_i)$$

## Optimization (Gradient Descent):

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

$$\mathbf{b} \leftarrow \mathbf{b} - \eta \cdot \frac{\partial L}{\partial \mathbf{b}}$$

Where:

- $\eta$  is the learning rate.

**Note:** The **Adam** optimizer (short for Adaptive Moment Estimation) is an advanced optimization algorithm that computes adaptive learning rates for each parameter by considering both *momentum* and *adaptive scaling* of the learning rate.

# Python Code for Classification of Iris dataset

```
# Import necessary libraries
import numpy as np
import tensorflow as tf
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input
from tensorflow.keras.utils import to_categorical
from sklearn.metrics import accuracy_score

# Step 1: Load the Iris dataset
iris = load_iris()
X = iris.data # Features (shape: 150 samples 4 features)
y = iris.target # Labels (integers 0, 1, 2 for 3 iris species)

# Step 2: Preprocess the data (standardization)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X) #Standardize features to have mean=0 and variance=1

# One-hot encode the target labels
y_one_hot = to_categorical(y, num_classes=3)
"""Convert integer labels like 2 into one-hot vectors for classification:
2      [0, 0, 1]"""

# Step 3: Split into train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y_one_hot, test_size=0.2, random_state=42
)
```

```
# Step 4: Define the MLP model
model = Sequential()
model.add(Input(shape=(4,))) # Input layer
model.add(Dense(10, activation='relu')) # Hidden layer
model.add(Dense(3, activation='softmax')) # Output layer

# Step 5: Compile the model
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
"""Adam adapts the learning rate for each parameter individually using first-order (mean) and
second-order (uncentered variance) moment estimates of the gradients."""

# Step 6: Train the model
model.fit(X_train, y_train, epochs=100, batch_size=10, verbose=1)

# Step 7: Evaluate the model
y_pred = model.predict(X_test)
y_pred_labels = np.argmax(y_pred, axis=1)
y_test_labels = np.argmax(y_test, axis=1)

accuracy = accuracy_score(y_test_labels, y_pred_labels)
print(f'Accuracy: {accuracy * 100:.2f}%')
```

# ANN Architecture for MNIST Digit Classification

- **Input Layer:** 784 neurons (flattened  $28 \times 28$  image)
- **Hidden Layer:** 128 neurons, ReLU activation
- **Output Layer:** 10 neurons, Softmax activation

**Hidden layer:**  $a^{(1)} = \text{ReLU}(W^{(1)}x + b^{(1)})$

$$W^{(1)} \in \mathbb{R}^{128 \times 784}, \quad b^{(1)} \in \mathbb{R}^{128}, \quad x \in \mathbb{R}^{784}$$

**Output layer:**  $\hat{y} = \text{softmax}(W^{(2)}a^{(1)} + b^{(2)})$

$$W^{(2)} \in \mathbb{R}^{10 \times 128}, \quad b^{(2)} \in \mathbb{R}^{10}, \quad \hat{y} \in \mathbb{R}^{10}$$

**Loss:**  $L = - \sum_{i=1}^{10} y_i \log(\hat{y}_i)$  (Categorical Crossentropy)

- **Update rule:**

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$
$$\mathbf{b} \leftarrow \mathbf{b} - \eta \cdot \frac{\partial L}{\partial \mathbf{b}}.$$

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Input
from tensorflow.keras.utils import to_categorical
from sklearn.metrics import accuracy_score

# Step 1: Load MNIST data
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Step 2: Normalize inputs
X_train = X_train / 255.0
X_test = X_test / 255.0

# Step 3: Reshape the data to fit into the neural network
X_train = X_train.reshape(-1, 28*28)
X_test = X_test.reshape(-1, 28*28)

# Step 4: One-hot encode labels
y_train_one_hot = to_categorical(y_train, num_classes=10)
y_test_one_hot = to_categorical(y_test, num_classes=10)
```

```
# Step 5: Define feedforward ANN with proper Input layer
model = Sequential()
model.add(Input(shape=(28*28,))) # Correct way to define input shape
model.add(Dense(units=128, activation='relu')) # Hidden layer
model.add(Dense(units=10, activation='softmax')) # Output layer

# Step 6: Compile model
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

# Step 7: Train model
model.fit(X_train, y_train_one_hot, epochs=10, batch_size=32, verbose=1)

# Step 8: Evaluate
y_pred = model.predict(X_test)
y_pred_labels = np.argmax(y_pred, axis=1)
acc = accuracy_score(np.argmax(y_test_one_hot, axis=1), y_pred_labels)
print(f'Accuracy: {acc * 100:.2f}%')
```

# Tensors in Deep Learning

- **Tensor:** Multidimensional array used in deep learning computations.
- **Types:**
  - 0D – Scalar
  - 1D – Vector (e.g., sequence of values)
  - 2D – Matrix (e.g., grayscale image)
  - 3D – Color image (height  $\times$  width  $\times$  RGB)
  - 4D – Video (sequence of 3D frames)
  - 5D – Collection of videos
- **Shape:** Tuple representing tensor dimensions.
- **Example:** 1-min 4K video @30 fps = 44.8B elements/tensor.
- **Challenge:** Tensors scale fast  $\rightarrow$  need efficient computation.
- **Solution:** Use GPUs or TPUs for tensor operations.

# Convolutional Neural Networks for Vision; Multi Classification with the MNIST Dataset

- **MNIST:** 70,000 grayscale digit images ( $28 \times 28$  pixels); values 0–255; labels 0–9.
- **Compared to Digits:** Higher resolution, more samples (Digits:  $8 \times 8$ , 0–16 scale, 1797 samples).
- **CNN (ConvNet):** Used for image recognition and beyond (e.g., NLP, recommender systems).
- **Output:** 10 class probabilities (0–9); highest = predicted digit.

# Basic Keras Neural Network

- **Model (Network):**
  - Sequence of layers (input, hidden, output).
  - Neurons process inputs using activation functions.
  - Input layer defines sample shape.
  - Hidden layers learn patterns from data.
  - Output layer produces final predictions.
  - Stacking more layers increases depth → deep learning.
- **Loss Function:**
  - Measures how well predictions match targets.
  - Lower loss  $\leftrightarrow$  better performance.
- **Optimizer:**
  - Minimizes loss function output.
  - Tunes network weights for improved accuracy.

# Loading the MNIST Dataset

- Use `tensorflow.keras.datasets.mnist` to load data:

```
from tensorflow.keras.datasets import mnist
(X_train, y_train), (X_test, y_test) = mnist.load_data()
```

- Returns training and testing sets as tuples: samples and labels.
- Downloads MNIST (28×28 grayscale images, digits 0–9).
- Keras runs on top of TensorFlow (`tensorflow.keras` vs `keras`).

# Visualizing MNIST Digits

- Enable plotting in notebooks: `%matplotlib inline`
- Import libraries:

```
import matplotlib.pyplot as plt
import seaborn as sns
sns.set(font_scale=2)
```

```
import numpy as np
index = np.random.choice(np.arange(len(X_train)), 24, replace=False)
figure, axes = plt.subplots(nrows=4, ncols=6, figsize=(16, 9))
for item in zip(axes.ravel(), X_train[index], y_train[index]):
    axes, image, target = item
    axes.imshow(image, cmap=plt.cm.gray_r)
    axes.set_xticks([]) # remove x-axis tick marks
    axes.set_yticks([]) # remove y-axis tick marks
    axes.set_title(target)
plt.tight_layout()
```

# Data Preparation for MNIST (Keras Convnet)

- Keras convnets require input shape: (width, height, channels).
- MNIST images are 28x28 pixels with 1 grayscale channel: (28, 28, 1).
- Reshape training and test data for convnet compatibility.

```
X_train = X_train.reshape((60000, 28, 28, 1))  
X_train.shape # Output: (60000, 28, 28, 1)  
  
X_test = X_test.reshape((10000, 28, 28, 1))  
X_test.shape # Output: (10000, 28, 28, 1)
```

# Data Normalization and One-Hot Encoding

- **Normalization:** Pixel values range from 0 to 255.
- Scale these values to the range [0.0, 1.0] for better training performance.

```
X_train = X_train.astype('float32') / 255.0  
X_test = X_test.astype('float32') / 255.0
```

- **One-Hot Encoding:** Convert class labels (integers 0–9) into 10-dimensional vectors.
- Only one position is 1.0 (corresponding to the class), others are 0.0.

```
from tensorflow.keras.utils import to_categorical  
  
y_train = to_categorical(y_train)  
y_test = to_categorical(y_test)  
  
print(y_train.shape) # (60000, 10)  
print(y_train[0]) # e.g., digit 5: [0. 0. 0. 0. 0. 1. 0. 0. 0. 0.]  
print(y_test.shape) # (10000, 10)
```

# Creating the CNN Model

- **Sequential Model:** A linear stack of layers where output of one layer is input to the next.
- **Feed-forward Network:** No cycles; used for image classification.

```
from tensorflow.keras.models import Sequential  
  
cnn = Sequential()
```

- **Import Layers:** We'll use convolution, pooling, flattening, and dense layers.

```
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten  
    , Dense
```

## Convolution Layer

- **Convolution Layer (Conv2D):** Learns features from small patches (kernels) of the input image.
- **Example:** A  $3 \times 3$  kernel slides over a  $6 \times 6$  image, producing a smaller output feature map.
- Convolution captures local spatial patterns and feeds them into deeper layers.

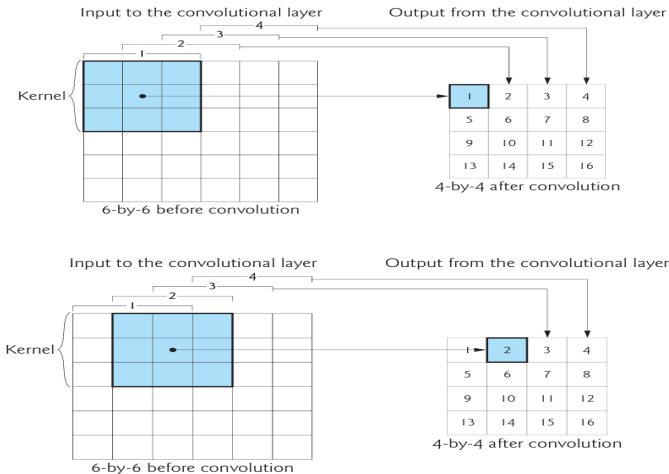
- **Kernel (Filter):** A small sliding window (e.g.,  $3 \times 3$ ) that moves across the image to extract features.
- Moves one pixel at a time (**stride = 1**) from left-to-right, top-to-bottom.
- Learns patterns like edges, lines, and curves based on neighboring pixels.
- Each filter generates a **feature map**—a 2D array of learned features.
- For MNIST ( $28 \times 28$ ), a  $3 \times 3$  kernel produces a  $26 \times 26$  output feature map.
- The number of filters (commonly 32, 64, etc.) determines the number of feature maps.

### Adding a Convolution layer:

```
from tensorflow.keras.layers import Conv2D

# Add convolution layer with 64 filters of size 3x3
cnn.add(Conv2D(filters=64,
                kernel_size=(3, 3),
                activation='relu',
                input_shape=(28, 28, 1)))
```

- `filters=64`: Number of filters (i.e., different feature extractors).
- `kernel_size=(3, 3)`: size of the kernel used in each filter
- `activation='relu'`: Applies ReLU non-linearity to learned features.
- `input_shape=(28, 28, 1)`: Required only for the first layer to define image shape.



## Dimensionality of the First Convolution Layer's Output:

- **First Conv Layer Output:** Input shape is (28, 28, 1); applying 64 filters of size (3, 3) results in output shape (26, 26, 64).
- Total features per image after this layer:  $26 \times 26 \times 64 = 43,264$
- This increase in dimensionality is significant and requires more computation.

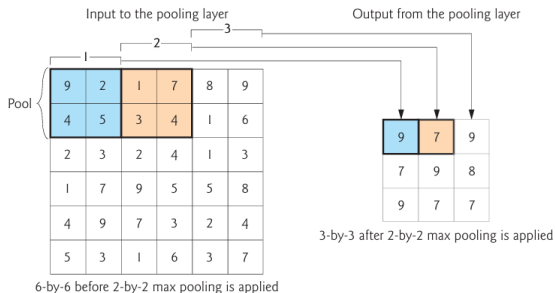
## Overfitting

- **Overfitting:** Happens when the model becomes too complex and memorizes training data.
- Performs well on training data but poorly on unseen data.
- Overfitting is common when feature maps become too large.
- Techniques to prevent overfitting:
  - Training for fewer epochs
  - Data augmentation
  - Dropout (discussed later)
  - L1/L2 regularization.

## Adding a Pooling Layer:

- Reduces dimensionality and computation, helping prevent overfitting.
- **Max Pooling:** Scans  $2 \times 2$  feature blocks and retains only the maximum value.
- Reduces output shape from (26, 26, 64) to (13, 13, 64)—a 75% reduction in features.
- Pool stride equals pool size (no overlap).

```
cnn.add(MaxPooling2D(pool_size=(2, 2)))
```



- **Alternative:** Larger-stride Conv2D layers can also reduce dimensionality without discarding features.

## Adding Another Convolution and Pooling Layer

- Convnets often stack more Conv and Pooling layers to learn complex features.
- We double the number of filters to 128 to capture more relationships.
- The second pooling reduces dimensionality again by 75%.

```
cnn.add(Conv2D(filters=128, kernel_size=(3,3), activation='relu'))  
cnn.add(MaxPooling2D(pool_size=(2, 2)))
```

- Output after Conv2D: (11, 11, 128); after pooling: (5, 5, 128)

## Flattening the Results:

- Flatten layer reshapes 3D output (5, 5, 128) into 1D (3200,).
- Required before feeding to Dense layers for classification.

```
cnn.add(Flatten())
```

### Adding a Dense Layer:

- Dense layers learn relationships among extracted features.
- 128 neurons process the 3200 features from the Flatten layer.

```
cnn.add(Dense(units=128, activation='relu'))
```

### Final Dense Layer for Classification:

- The output layer contains 10 neurons—one for each digit (0–9).
- Uses softmax activation to produce classification probabilities.

```
cnn.add(Dense(units=10, activation='softmax'))
```

### Printing the Model's Summary:

- `model.summary()` displays each layer's output shape and parameter count.
- None in output shapes indicates dynamic batch size.
- All parameters are trainable unless explicitly frozen (used in transfer learning).

```
cnn.summary()
```

## Visualizing a Model's Structure:

- Visualize the model using `plot_model` from `tensorflow.keras.utils`.
- The model diagram is saved as `convnet.png` and displayed with IPython's `Image`.

```
from tensorflow.keras.utils import plot_model
from IPython.display import Image

plot_model(cnn, to_file='convnet.png',
           show_shapes=True, show_layer_names=True)
Image(filename='convnet.png')
```

## Compiling the Model:

- Compile the model with an optimizer, a loss function, and evaluation metrics.
- `optimizer='adam'`: Efficient and widely used for many models.
- `loss='categorical_crossentropy'`: Suitable for multi-class classification.
- `metrics=['accuracy']`: Monitors classification accuracy.

```
cnn.compile(optimizer='adam',  
            loss='categorical_crossentropy',  
            metrics=['accuracy'])
```

'adam':

- Adam adjusts learning rates for each parameter automatically.
- It combines momentum and RMSProp to improve updates.
- It updates weights using gradient descent with adaptive steps.

## Training the model:

- Train the model using `fit` with training data and one-hot encoded labels.
- `epochs=5`—Model will iterate over the training set 5 times.
- `batch_size=64`—Processes 64 samples at a time during training.
- `validation_split=0.1`—Reserves 10% of training data for validation.

```
cnn.fit(X_train, y_train,  
        epochs=5, batch_size=64,  
        validation_split=0.1)
```

## Evaluating the Model and Making Predictions:

- Evaluate accuracy on unseen data using the evaluate method.
- predict returns the class probabilities for the test samples.
- Index with highest probability in predictions[0] corresponds to predicted digit.

```
loss, accuracy = cnn.evaluate(X_test, y_test)
print("Test accuracy:", accuracy)

predictions = cnn.predict(X_test)
print("True label:", y_test[0])

for index, probability in enumerate(predictions[0]):
    print(f'{index}: {probability:.10%}')
```

# QUESTION:

Find Convolution, ReLU, and Max Pooling of the following data:

- Input Image ( $4 \times 4$ ):

```
[[1, 2, 0, 1],  
 [3, 1, 2, 2],  
 [1, 0, 1, 3],  
 [2, 1, 2, 1]]
```

- Filter/Kernel ( $2 \times 2$ ):

```
[[ 1,  0],  
 [ 0, -1]]
```

- Use stride = 1.

# ANSWER:

## Convolution Operation

- The figure shows the convolution filter at its first position (top-left  $2 \times 2$ ).
- The blue region highlights the part of the input currently being processed.
- The filter is shown on the right.

|   |   |   |   |
|---|---|---|---|
| 1 | 2 | 0 | 1 |
| 3 | 1 | 2 | 2 |
| 1 | 0 | 1 | 3 |
| 2 | 1 | 2 | 1 |

**$4 \times 4$  Input Image**

|   |    |
|---|----|
| 1 | 0  |
| 0 | -1 |

**$2 \times 2$  Filter**

- Dot product:  $1 \times 1 + 2 \times 0 + 3 \times 0 + 1 \times (-1) = 0$

- Perform  $2 \times 2$  convolution with stride 1:

|        |               |                                 |
|--------|---------------|---------------------------------|
| (0,0): | [[1,2],[3,1]] | $1*1 + 2*0 + 3*0 + 1*(-1) = 0$  |
| (0,1): | [[2,0],[1,2]] | $2*1 + 0*0 + 1*0 + 2*(-1) = 0$  |
| (0,2): | [[0,1],[2,2]] | $0*1 + 1*0 + 2*0 + 2*(-1) = -2$ |
| (1,0): | [[3,1],[1,0]] | $3*1 + 1*0 + 1*0 + 0*(-1) = 3$  |
| (1,1): | [[1,2],[0,1]] | $1*1 + 2*0 + 0*0 + 1*(-1) = 0$  |
| (1,2): | [[2,2],[1,3]] | $2*1 + 2*0 + 1*0 + 3*(-1) = -1$ |
| (2,0): | [[1,0],[2,1]] | $1*1 + 0*0 + 2*0 + 1*(-1) = 0$  |
| (2,1): | [[0,1],[1,2]] | $0*1 + 1*0 + 1*0 + 2*(-1) = -2$ |
| (2,2): | [[1,3],[2,1]] | $1*1 + 3*0 + 2*0 + 1*(-1) = 0$  |

- Final Convolution Output:

```
[[ 0,  0, -2],  
 [ 3,  0, -1],  
 [ 0, -2,  0]]
```

## ReLU Activation:

- Apply ReLU:  $\text{ReLU}(x) = \max(0, x)$

Before ReLU:

```
[[ 0,  0, -2],  
 [ 3,  0, -1],  
 [ 0, -2,  0]]
```

After ReLU:

```
[[0, 0, 0],  
 [3, 0, 0],  
 [0, 0, 0]]
```

## Max Pooling Operation with Stride 2:

- **Max Pooling** reduces dimensionality and helps prevent overfitting.
- We use a **2×2 filter** with **stride = 2**, meaning the pooling windows do not overlap.
- For each 2×2 window, the maximum value is retained while others are discarded.

Input (ReLU Output):

```
[[0, 0, 0],  
 [3, 0, 0],  
 [0, 0, 0]]
```

Step 1: Define 2x2 windows with stride 2:

Only one window fits:

```
[[0, 0],  
 [3, 0]]    max = 3
```

No other 2x2 window fits in a 3x3 matrix with stride 2.

- **Final Max Pooled Output (1×1):**

```
[[3]]
```